

Dynamic Programming & Reinforcement Learning: Assignment 3

Vrije Universiteit Amsterdam - Group 14 - 08/12/2023

X
1

Ángel A. Gil
2788988

1 Introduction

This report presents the design and implementation of a Monte Carlo Tree Search (MCTS) algorithm tailored to outperform a random agent in the game of Connect 4.

Firstly, we describe the problem, detailing the initial state and the structure of the Connect4 class. This is followed by an in-depth examination of our MCTS algorithm, elaborating on its exploration and update mechanisms. Subsequently, we provide a detailed analysis of a game, comparing theoretical winning probabilities with those estimated by our MCTS. The report concludes with statistical evaluations, offering insights into the convergence process of our approach.

Disclaimer: The code submitted alongside this report was developed independently with consultative support from ChatGPT-4V. While the LLM did not significantly contribute to resolving complex aspects of the problem, it aided in refining the overall structure and expediting the styling of our implementation.

2 Problem Description and encoding

Connect Four is a two-player board game, where the players choose a color and then take turns dropping colored discs into a seven-column, six-row vertically suspended grid. The pieces fall straight down, occupying the lowest available space within the column. The objective of the game is to be the first to form a horizontal, vertical, or diagonal line of four of one's own discs.

In this assignment, we are given an already ongoing game, shown in Figure 1, where only 2 actions are possible: to either play columns 1 or 7. Our agent conducts the dark colour and is the one to make the next move.

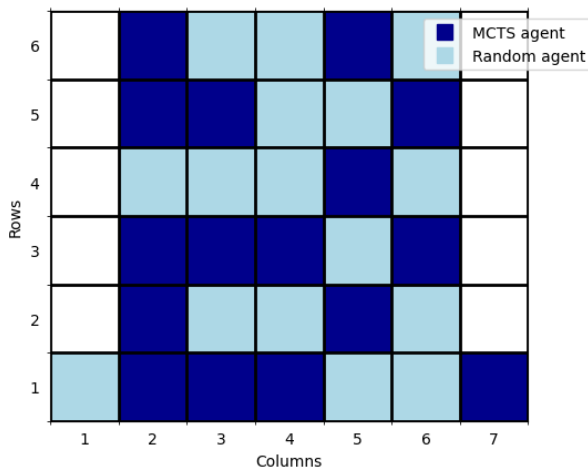


Figure 1: Initial state of the game. MCTS agent to play

We implemented a Connect4 class with the following methods:

- `__init__`: Initializes the game board with the given state, sets board dimensions (6x7), and designates the first player to move.
- `__str__`: Returns a string representation of the game board, for ease of debugging and fast visualization purposes while working on the MCTS.
- `get_possible_actions`: returns the list of columns that are not yet full.
- `is_terminal`: Checks if the game has reached a terminal state (win, loss, or draw) and returns a boolean status along with the game outcome (1, -1, or 0).
- `check_winner`: Determines if the specified player has won by checking for four consecutive discs horizontally, vertically, or diagonally.
- `play`: Executes a move (dropping a disc in a chosen column), checks for game termination, and switches the player turn.
- `get_next_state`: Generates and returns a new game state resulting from playing a specified move, ensuring the original state remains unchanged (deep copy).

Additionally, a Connect4Visualizer class was developed, capable of generating images like Figure 1. This class features a dynamically adaptive title indicating the next player's turn and automatically recognizes a player's victory, highlighting the winning sequence in yellow.

3 Our MCTS implementation

MCTS is grounded in the balance between exploration of uncharted moves and exploitation of known advantageous ones. We iteratively built a search tree: nodes were expanded based on game state simulations, with the UCB formula guiding the selection process. The algorithm's essence lies in its adaptability and heuristic-free nature, which we tailored for Connect 4's unique state space.

Our implementation encodes the Connect 4 board as a two-dimensional array, with actions represented by available columns for disc placement. Terminal states are identified by either a win, loss, or draw, determined by a sequence of checks for four contiguous discs.

4 Average expected reward: Theory vs MCTS

Visualizations play a pivotal role in elucidating the decision-making process of our MCTS implementation. Figures will be included to illustrate the progression of the search tree and the varying probabilities of winning associated with each possible action over time.

5 Insights on the algorithm's convergence

The convergence of the MCTS algorithm is captured through plots depicting the stabilization of winning probabilities as the number

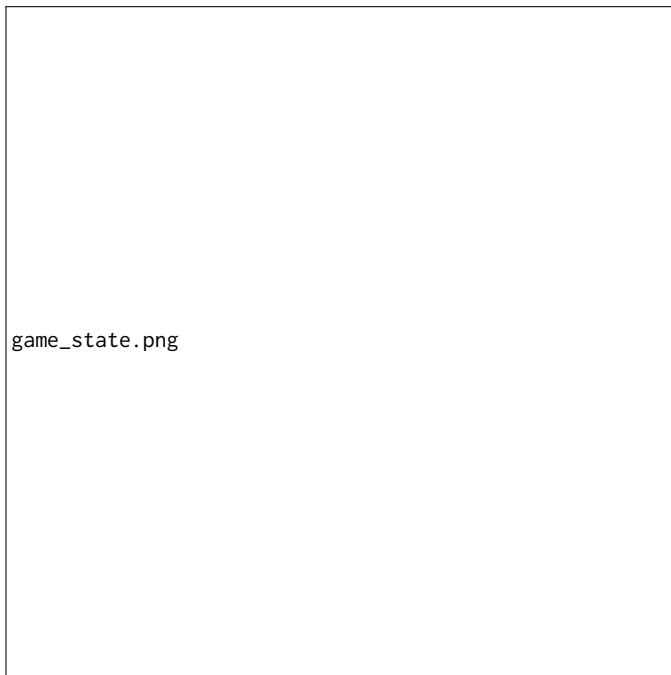


Figure 2: Connect 4 game state encoding.



Figure 3: Visualization of the MCTS decision-making process.

of simulations increases, providing insights into the algorithm's performance and efficacy.

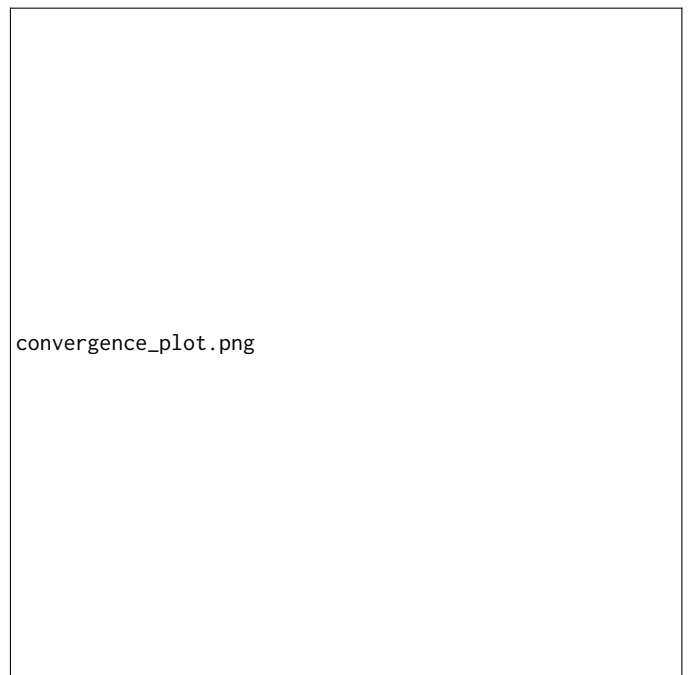


Figure 4: Convergence of the MCTS algorithm over multiple games.

6 Conclusion

Optional.